Desenvolvimento da Biblioteca Geometria em C

Integrantes: Davi Mendonça, Leandro Takara, Marcelo Zhang e Marcos Akira

1 Introdução

O objetivo do trabalho é desenvolver uma biblioteca em linguagem C que resolva problemas envolvendo geometria em $\mathbb{R}^2$, encontrados em situações cotidianas e problemas matemáticos.

Para isso, desenvolvemos 4 módulos para esse domínio, sendo eles: Geometria Plana, Polígono, Envoltória Convexa e Interseção e Proximidade, que foram distribuídos para cada membro da equipe.

2 Arquitetura do Software

A arquitetura do trabalho é dividida em algumas partes dentro do repositório no GitHub, que iremos tratar uma por uma. Além disso, será abordado, nesta seção, a relação existente entre os módulos criados.

2.1 Divisão do Projeto

O projeto apresenta algumas seções separadas, que realizam suas respectivas funções:

2.1.1 app: diretório criado para aplicação cliente, com o arquivo “main.c”, “svg.c” e “svg.h” que utilizarão os recursos criados no trabalho.

2.1.2 include: diretório criado para armazenar a interface pública dos módulos criados, ou seja, os arquivos “.h”, onde se encontra apenas as funções declaradas, sem detalhes. Em “include”, teremos os arquivos “general.h”, “geometry2d.h”, “polygon.h”, “convex_hull.h” e “spatial.h”.

2.1.3 src: diretório criado para armazenar a implementação dos módulos, ou seja, os arquivos “.c”, onde os códigos detalhados se encontram. Em “src”, teremos os arquivos “general.c”, “geometry2d.c”, “polygon.c”, “convex_hull.c” e “spatial.c”.

2.1.4 tests: diretório criado para armazenar os testes criados para o projeto, que conterá os arquivos para testar as funções criadas de cada módulo, além do arquivo “test_utils.h”, que fornece algumas utilidades essenciais para os testes.

2.1.5 .gitignore: arquivos criados para listar os artefatos de compilação a ser ignorados pelo Git, como diretórios “build/”, “lib/” e “bin/”.

2.1.6 Makefile: arquivo criado como base para compilação do projeto, definindo as regras e as opções para Make, mais detalhes na seção 5.

2.1.7 README.md: arquivo com manual de instruções e a apresentação oficial do projeto.

2.2 Dependência dos Módulos

Para facilitar a visualização das dependências, iremos criar algumas subseções para cada módulo e os respectivos módulos de dependência.

2.2.1 general: o módulo “general” é o módulo base, então não depende de nenhum outro módulo.

2.2.2 geometry2d: o módulo “geometry2d” depende apenas do módulo “general”, uma vez que exige apenas o uso de pontos e segmentos de reta.

2.2.3 polygon: o módulo “polygon” depende dos módulos “general” e “geometry2d” para manipulação dos polígonos.

2.2.4 convex_hull: o módulo “convex_hull” depende dos módulos “general”, “geometry2d” e “polygon”.

2.2.5 spatial: o módulo “spatial” depende dos módulos “general”, “geometry2d” e “polygon” análise de proximidade e interseção entre objetos geométricos.

3 Módulos

Apesar de o trabalho ser dividido em 4 módulos centrais, foi criado um quinto módulo auxiliar, com funções e estruturas que serão utilizadas por todos os quatro módulos.

3.1 general

O módulo “general” é a base para todo o trabalho, ele contém apenas duas estruturas, com suas respectivas funções de criação.

3.1.1 Ponto: a estrutura ponto recebe dois valores do tipo “double” para representar a posição de um ponto em um plano cartesiano usual. A sua criação é feita a partir da função create_point();

3.1.2 Segmento: a estrutura segmento recebe duas estruturas ponto, que serão os pontos que definirão um segmento de reta em um plano cartesiano usual. A sua criação é feita a partir da função create_segment();

3.2 geometry2d

O módulo “geometry2d” é implementado para Geometria Plana, contendo 5 funções principais e utiliza das estruturas e das funções do módulo “general”

3.2.1 point_on_segment: a função recebe ponteiro de uma estrutura Segmento e um ponteiro de uma estrutura Ponto, retornando valor “1” caso o ponto está sobre o segmento de reta e “0” caso contrário.

A função identifica a sobreposição a partir de dois passos, verifica se o ponto está dentro da caixa envolvente do segmento e verifica se o ponto está sobre a reta definida pelo segmento de reta. Se algum ponteiro for nulo, a função retorna 0. Se os dois pontos do segmento forem iguais, a função retorna 1 apenas quando o ponto coincidir com eles.

3.2.2 point_distance: a função recebe dois ponteiros de dois pontos diferentes e retorna o valor, do tipo “double”, da distância entre eles. O método utilizado para extrair esse valor é justamente a aplicação da fórmula da distância. Se algum ponteiro for nulo, a função retorna -1.

3.2.3 point_line_distance: a função recebe dois ponteiros, um segmento de reta e um ponto, e calcula o valor da distância, do tipo double, entre a reta infinita definida pelo segmento de reta e o ponto.

A função gera a equação da reta a partir do segmento fornecido e aplica a fórmula de distância entre reta e ponto. Se algum ponteiro for nulo ou se os dois pontos do segmento coincidirem, caso em que o segmento não define uma reta, a função retorna -1.

3.2.4 line_intersection: a função recebe dois ponteiros de dois segmentos de reta, que definirão duas retas infinitas, e retorna uma estrutura Ponto que será as coordenadas do ponto de intersecção entre as duas retas.

A função gera as equações de reta definidas pelos dois segmentos e aplica a regra de Cramer para obter o ponto de intersecção. Se as retas forem paralelas, se algum ponteiro for nulo ou se algum segmento tiver os dois pontos iguais, a função retorna o ponto (NAN, NAN), que pode ser identificado com a função isnan, da biblioteca <math.h>.

3.2.5 lines_position: a função recebe dois ponteiros de dois segmentos de reta que definirão duas retas infinitas, retornando PARALELOS, PERPENDICULARES ou CONCORRENTES, definidos como 0, 1 e 2, respectivamente, no macro, para cada caso possível de ocorrência. A definição das mensagens foi feita no arquivo de cabeçalho “geometry2d.h”.

A identificação ocorre a partir da análise do produto escalar e a proporcionalidade entre os vetores geradores das linhas, e retorna INVALIDO (-1) caso algum ponteiro seja nulo ou algum segmento tenha os dois pontos iguais

3.3 polygon

O módulo “polygon” é destinado para manipulação de polígonos sobre $\mathbb{R}^2$, possuindo 6 funções centrais, 2 funções bases e 1 estrutura e utiliza as estruturas e as funções presentes nos módulos “general” e “geometry2d”.

3.3.1 Poligono: a estrutura polígono é essencial para funcionamento do módulo atual e dos módulos “convex_hull” e “spatial”. Ela recebe um inteiro que representa a quantidade de vértices do polígono e um array de pontos. A sua criação é feita por meio da função “create_polygon”.

3.3.2 create_polygon: a função recebe um inteiro n, que indica a quantidade de vértices do polígono, e um ponteiro para array de pontos. Com isso, a função retorna estrutura Poligono. Se n for menor que 3, se o vetor de vértices for nulo ou se a alocação de memória falhar, a função retorna um polígono vazio (n igual a 0 e vértices iguais a NULL). Os vértices são copiados, portanto o vetor original pode ser alterado ou liberado depois. Além disso, os vértices consecutivos do array devem ser adjacentes no polígono.

3.3.3 free_polygon: a função recebe ponteiro de um polígono e libera a memória utilizada pelo array de vértices, zerando o índice n, que representa quantidade de vértices, e atribui NULL ao ponteiro dos vértices. Se o ponteiro recebido for nulo, a função não faz nada.

3.3.4 signed_area: a função recebe ponteiro de um polígono para calcular a sua área, retornando 0 caso o polígono for inválido (NULL), os vértices forem inválidos ou a quantidade de vértices for inferior a 3.

O cálculo de área é feito a partir da fórmula de Gauss, somando o determinante da matriz formada pelas coordenadas dos vértices adjacentes e dividindo o total por 2. O resultado pode aparecer em sinal positivo, caso o polígono esteja representado em sentido anti-horário, e sinal negativo caso contrário.

3.3.5 polygon_area: a função recebe ponteiro de um polígono e retorna o valor, sempre positivo, da sua área. A função invoca a função “signed_area” e extrai o valor absoluto a partir da função “fabs”, da biblioteca <math.h>.

3.3.6 polygon_perimeter: a função recebe ponteiro de um polígono e calcula o valor de seu perímetro. Como os vértices dos polígonos estão ordenados de tal forma que os vértices adjacentes do array são vértices adjacentes do polígono, então o perímetro é dado com a soma das distâncias entre dois pontos adjacentes do array de vértices, por meio da função “point_distance”.

3.3.7 polygon_centroid: a função recebe ponteiro de um polígono e retorna a estrutura Ponto, correspondente ao centróide do polígono, caso o polígono ou o array dos vértices sejam inválidos, retorna ponto (0, 0), assim como para quantidade

de vértice menores que 3. Para encontrar o centroide, a função aplica a fórmula de centroide, fazendo o uso da função “signed_area”.

3.3.8 is_convex: a função recebe ponteiro de um polígono e retorna inteiro 1, caso o polígono seja convexo, 0, caso contrário e caso o polígono inserido for inválido.

A função pega todos os trios de pontos adjacentes e calcula o produto vetorial e compara com a orientação do polígono, caso o resultado do produto e a orientação apresentarem sinais opostas, então o polígono não será convexo.

3.3.9 polygon_contains_point: a função recebe ponteiros de polígono e ponto, e retorna 1 caso o ponto estiver dentro do polígono ou sobre as suas arestas, 0, caso contrário e caso o ponto e o polígono sejam inválidos.

A função realiza o teste para verificar se o ponto está sobre alguma aresta do polígono, caso esteja, então o ponto está dentro do polígono. Após isso, gera-se uma reta contínua para a direita do ponto, calculando a quantidade de interseção existente entre essa reta e o polígono, caso o número de interseção seja ímpar, então o ponto se encontra dentro do polígono.

3.4 convex_hull

O módulo “convex_hull” é módulo destinado para análise de fecho convexo dos polígonos, apresentando 5 funções centrais e 4 funções auxiliares e utiliza as estruturas e as funções presentes nos módulos “general”, “geometry2d” e “polygon”.

3.4.1 cross: a função auxiliar recebe três ponteiros de três pontos e retorna o resultado do produto vetorial entre eles.

3.4.2 dist2: a função auxiliar recebe dois ponteiros de dois pontos e retorna o valor da distância ao quadrado entre eles.

3.4.3 vazio: a função auxiliar não recebe parâmetro e retorna um polígono vazio, isto é, com 0 vértices e com array de vértices igual a NULL.

3.4.4 menor_ponto: a função auxiliar recebe dois ponteiros de dois pontos, a e b, e retorna 1, caso o ponto a seja menor que ponto b, e 0 caso contrário. O critério privilegia o menor valor entre as coordenadas em y do ponto, para caso de empate, o critério parte para menor valor entre as coordenadas em x.

3.4.5 convex_hull_jarvis: a função recebe um inteiro n, a quantidade de pontos passados, e ponteiro de array com n pontos e retorna uma estrutura Poligono que representa o fecho convexo dos pontos passados como parâmetro. A busca pelo fecho convexo se dá pelo algoritmo de Jarvis, que realiza as seguintes operações

- Validação dos parâmetros, retornando polígono vazio caso o array dos pontos for inválido ou caso a quantidade de pontos for inferior a 3.
- Alocação de memória para array “fecho”, retornando polígono vazio, com função auxiliar “vazio”, caso a alocação falhe.
- Encontra o menor ponto dentre os pontos do array, a partir do critério definido na função auxiliar “menor_ponto”, adiciona no array “fecho” e define como ponto atual.
- Percorre todos os pontos e busca pelo candidato para o fecho, tal que seja diferente do ponto atual.
- Faz produto vetorial, com a função auxiliar “cross”, entre os pontos candidato, atual e candidato anterior, e busca pelo ponto que está mais à direita, entre os demais.
- Adiciona este ponto no array “fecho” e o define como novo atual.
- Repete o processo até que retorne ao ponto inicial (o menor) ou quando a quantidade de pontos no array “fecho” for maior que a quantidade total de vértices.
- Após completar o fecho, forma-se o polígono e limpa a memória alocada para array “fecho”.

O algoritmo de Jarvis apresenta complexidade de tempo na ordem de O(n.m), para m a quantidade de vértices do polígono final.

3.4.6 convex_hull_graham: a função recebe um inteiro n, a quantidade de pontos passados, e ponteiro de array com n pontos e retorna uma estrutura Poligono que representa o fecho convexo dos pontos passados como parâmetro. A busca pelo fecho convexo se dá pelo algoritmo de Graham. A função apresenta uma função e uma variável estática auxiliar, que, por serem estáticas, só são visíveis dentro de “convex_hull.c”, sendo elas

- Variável estática “pivo”, sendo referência para calcular ângulo dos demais vértices.
- Função estática “cmp”, que realiza a ordenação dos pontos, em relação a ângulo formado com pivô (anti-horário), com critério de desempate a distância.

Com isso, a função “convex_hull_graham” realiza as operações abaixo

- Validação dos parâmetros, retornando polígono vazio caso o array dos pontos for inválido ou caso a quantidade de pontos for inferior a 3.
- Alocação de memória para dois vetores, “v” e “pilha”, retornando polígono vazio, a partir da função auxiliar “vazio”, caso ocorra falha de alocação.
- Copia os pontos passados como parâmetro para o vetor “v”.

- Define o menor ponto, a partir do critério da função “menor_ponto” e troca a posição do menor ponto encontrado em “v” com o primeiro ponto, isto é, realizar troca entre “v[menor]” e v[0].
- Realizar a ordenação por função “qsort”, do módulo <stdlib.h>, do array “v”, definindo como pivô o menor ponto, ou o primeiro ponto, a partir dos critérios definidos pela função “cmp”.
- Declaração da variável k, como sendo a posição no array “pilha”.
- Percorre o array “v”, adicionando os elementos na pilha e realizando contagem de k. Quando k for igual ou superior a 2, analisamos os dois últimos pontos da pilha, removendo o último ponto da pilha enquanto os três pontos forem colineares ou fizerem curva à direita. Cópias repetidas do pivô também são descartadas.
- Ao fim do processo, cria-se o polígono com os pontos do array “pilha”, com k vértices e libera-se a memória dos vetores “v” e “pilha”

O algoritmo de Graham apresenta complexidade de tempo na ordem de O(n log n). Essa diferença de complexidade de tempo entre Jarvis e Graham permite a escolha de um algoritmo mais eficiente a depender dos parâmetros passados.

3.4.7 convex_hull_area: a função recebe um inteiro n, a quantidade de pontos, e ponteiro de um array com n pontos e retorna a área do fecho convexo formado por esses pontos.

A “convex_hull_area” faz chamada à função “convex_hull_graham” para obter o fecho convexo, para depois chamar a função “polygon_area”, que gera o valor da área do fecho. Ao fim do processo, a memória do fecho é liberada a partir da função “free_polygon”.

3.4.8 convex_hull_perimeter: a função recebe um inteiro n, a quantidade de pontos, e ponteiro de um array com n pontos e retorna o perímetro do fecho convexo formado por esses pontos.

A “convex_hull_perimeter” faz chamada à função “convex_hull_graham” para obter o fecho convexo, para depois chamar a função “polygon_perimeter”, que gera o valor do perímetro do fecho. Ao fim do processo, a memória do fecho é liberada a partir da função “free_polygon”.

3.4.9 convex_hull_diameter: a função recebe um inteiro n, a quantidade de pontos, e ponteiro de um array com n pontos e retorna o valor do diâmetro.

O diâmetro é calculado sobre o fecho obtido por “convex_hull_graham”: a função compara as distâncias entre todos os pares de vértices do fecho, já que os dois pontos mais distantes do conjunto são sempre vértices do fecho. Para entradas inválidas, ou sem fecho, retorna 0.

3.5 spatial

O módulo “spatial” é módulo implementado para análise de Proximidade e Interseção de objetos geométricos, possui 4 funções públicas e 7 funções auxiliares, declaradas como static e usadas apenas dentro de “spatial.c” e utiliza as estruturas e as funções presentes nos módulos “general”, “geometry2d” e “polygon”.

3.5.1 closest_pair: a função recebe um inteiro n, a quantidade de pontos, e um ponteiro para o array de pontos, nessa ordem, como nas funções do módulo “convex_hull”. A partir disso, a função retorna o par (se existir mais de um, retorna qualquer um) de pontos mais próximos um do outro, caso não exista, retorna-se NULL se a array tiver menos de 2 pontos, o ponteiro for nulo ou quando a alocação de memória para array falhar.

A busca pelo par é feita a partir da comparação entre distância de todos os pares possíveis a serem formados no array. Ao fim da função, retorna-se um array, de tamanho 2, com os dois pontos. O array retornado deve ser liberado com free.

3.5.2 intersection_polygon: a função recebe dois polígonos, A e B, e ponteiro para inteiro, que indicará o tamanho do array criado com os pontos de interseção entre os polígonos. A função executa as seguintes operações:

- Cria um array, a partir de malloc, de tamanho 10 para armazenar os pontos de interseção, retornando NULL caso a criação falhe;
- Gera todos os segmentos de reta que formam os lados do polígono A e compara com os lados do polígono B, a partir da função “create_segment”. A comparação é feita analisando se são paralelas, extraindo o ponto de interseção e analisa se o ponto de interseção está sobre os segmentos de reta, por meio das funções “lines_position” e “line_intersection”;
- Verifica se o ponto já foi adicionado no array, fazendo comparação a partir da precisão GEOM_EPS;
- Caso nenhum ponto for adicionado, retorna NULL;
- Alocação de mais memória para array, a partir de realloc, caso a quantidade de pontos exceda do limite estabelecido anteriormente. Caso a alocação falhe, retorna NULL;
- Realocação da memória do array, fixando a quantidade necessária para todos os pontos de interseção e retorna o array. Caso a realocação falhe, retorna NULL.
- Retorna NULL, com length igual a 0, se algum polígono for inválido. Se o próprio length for nulo, retorna NULL sem escrever nele. O array retornado deve ser liberado com free.

3.5.3 intersection_region: a função recebe dois polígonos, A e B, e ponteiro de inteiro “length” para indicar quantidade de regiões formados pela interseção entre A e B. A função retorna um array de polígonos que são as regiões de interseção entre A e B. A função realiza as seguintes operações:

- Valida as entradas: retorna NULL, com length igual a 0, se algum polígono for inválido. Se o próprio length for nulo, retorna NULL sem escrever nele.
- Se A está contido em B, ou B em A, a região de interseção é uma cópia do polígono contido.
- Obtém os pontos de interseção com a função "intersection_polygon". Sem pontos de interseção, retorna NULL.
- Cria os polígonos "xPolygonA" e "xPolygonB" com a função auxiliar "add_intersection_points", que insere os pontos de interseção como vértices de A e de B.
- Cria o array "used", que marca os pontos de interseção já percorridos, e o array "regions", com 5 posições iniciais e realloc quando necessário.
- Para cada ponto de interseção ainda não usado, contorna uma região. A partir do ponto, percorre a borda de A pelo trecho que está dentro de B, com a função auxiliar "find_points_path", até chegar ao próximo ponto de interseção. Dali passa para a borda de B, pelo trecho que está dentro de A. A função alterna entre os dois polígonos a cada ponto de interseção, até voltar ao ponto de partida.
- O sentido do percurso é escolhido pelo ponto médio da aresta. Se o ponto médio entre o ponto atual e o vértice seguinte estiver dentro do outro polígono, o percurso segue para a frente (1); se o ponto médio com o vértice anterior estiver, segue para trás (-1, que prevalece). O ponto médio é usado porque o vértice vizinho pode ser também um ponto de interseção: nesse caso ele está sobre a borda do outro polígono e seria considerado dentro, mesmo que a aresta entre os dois esteja fora.
- Pontos em que os polígonos apenas se tocam não fecham uma região e são descartados.
- No fim, o array "regions" é realocado para o número exato de regiões e retornado. Cada polígono do array deve ser liberado com "free_polygon", e o array com "free".

Limitação: quando um vértice de um polígono está exatamente sobre a borda do outro, ou quando há arestas sobrepostas, a região pode não ser encontrada.

3.5.4 is_polygon_inside: a função recebe dois polígonos, A e B, e retorna o inteiro 1 caso todos os vértices de A estejam dentro de B ou sobre a sua borda, e 0 caso contrário ou caso algum dos polígonos seja inválido.

Primeiro a função valida se ambos os polígonos são válidos e depois analisa cada vértice do polígono A e verifica se ele está contido em B, a partir da função “polygon_contains_point”. Como apenas os vértices são verificados, o resultado só é garantido quando B é convexo. Se B for côncavo, uma aresta de A pode sair de B mesmo com todos os vértices dentro dele.

3.5.5 add_intersection_points: função auxiliar, usada por “intersection_region”. A função recebe dois polígonos, A e B, retornando um polígono que será cópia de A, chamado “xPolygonA”, em que os pontos de interseção entre A e B são inseridos como vértices de A.

O procedimento de identificação de ponto de interseção segue semelhante à função “intersection_polygon”, mas com ordenação dos pontos a partir de Bubble Sort pela distância ao vértice inicial da aresta analisada. Dessa forma, os vértices de “xPolygonA” ficam na ordem em que são percorridos pela borda.

4 API pública e Aplicação Cliente

A API pública do projeto é formada apenas pelos arquivos de cabeçalho presente no diretório “include/”, sendo eles: “general.h”, “geometry2d.h”, “polygon.h”, “convex_hull.h” e “spatial.h”. A aplicação cliente tem apenas o acesso a esses arquivos de cabeçalho, sem acesso aos detalhamentos presentes em arquivos “.c”.

4.1 API pública

Como visto na seção 3, os módulos apresentam as funções e as estruturas próprias, mas algumas delas são destinadas apenas para os arquivos “.c”. Por isso, listamos as funções de API pública.

4.1.1 Tipos e Constantes

- **Ponto (general.h);**
- **Segmento (general.h);**
- **Poligono (polygon.h);**
- **GEOM_EPS (general.h);**
- **INVALIDO (geometry2d.h);**
- **PARALELOS (geometry2d.h);**
- **PERPENDICULARES (geometry2d.h);**
- **CONCORRENTES (geometry2d.h).**

4.1.2 Funções

- **create_point (general.h);**
- **create_segment (general.h);**

- **point_on_segment (geometry2d.h);**
- **point_distance (geometry2d.h);**
- **point_line_distance (geometry2d.h);**
- **line_intersection (geometry2d.h);**
- **lines_position (geometry2d.h);**
- **create_polygon (polygon);**
- **free_polygon (polygon);**
- **signed_area (polygon);**
- **polygon_area (polygon);**
- **polygon_perimeter (polygon);**
- **polygon_centroid (polygon);**
- **is_convex (polygon);**
- **polygon_contains_point (polygon);**
- **convex_hull_graham (convex_hull);**
- **convex_hull_jarvis (convex_hull);**
- **convex_hull_area (convex_hull);**
- **convex_hull_perimeter (convex_hull);**
- **convex_hull_diameter (convex_hull);**
- **closest_pair (spatial);**
- **intersection_polygon (spatial);**
- **intersection_region (spatial);**
- **is_polygon_inside (spatial).**

Os detalhes de cada função e estrutura podem ser encontrados na seção 3.

4.2 Aplicação Cliente

A aplicação cliente está localizada no diretório “app/” e é composta por arquivos “main.c”, “svg.c” e “svg.h”. O seu objetivo é demonstrar as funcionalidades públicas da biblioteca “libgeom”, desenvolvido pelo grupo. Além de mostrar os resultados no terminal, a aplicação gera representação gráfica em SVG, a partir dos arquivos “svg.c” e “svg.h”, para facilitar a visualização dos exemplos geométricos.

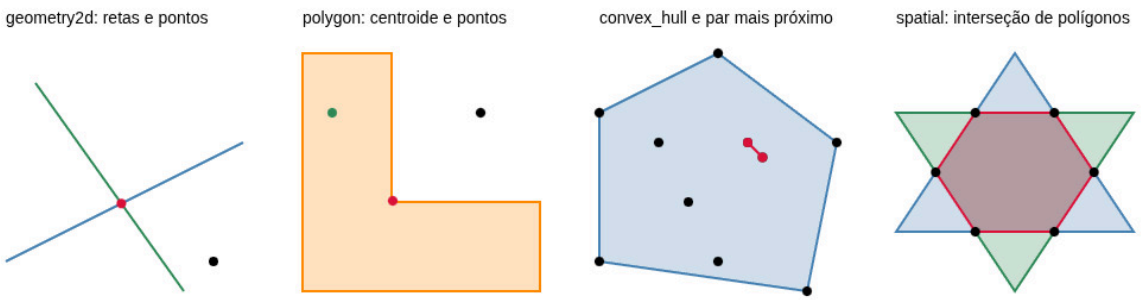
A função “main”, do arquivo “main.c” recebe como argumento de linha de comando o caminho do arquivo SVG. Se nenhum caminho for informado, usa “demo.svg”, e o Makefile passa “bin/demo_static.svg” e “bin/demo_shared.svg”. Se o arquivo não puder ser criado, a aplicação mostra uma mensagem de erro e termina com código 1. Com a presença de arquivo SVG, a função “main” inicia o arquivo, a partir da função “svg_begin()”, para execução das funções de demonstração. Ao fim da execução, a função “svg_end()” encerra o documento e fecha o arquivo.

Sejam as funções executadas e as suas utilizados abaixo:

- **demo_geometry2d:** cria duas retas e um ponto, classifica a posição relativa das retas, calcula a interseção das retas, quando aplicável, verifica se essa interseção pertence a um segmento e calcula distâncias.
- **demo_polygon:** cria um polígono côncavo em forma de L, informa área, perímetro, centróide, convexidade e testes de contenção para dois pontos.
- **demo_convex_hull:** calcula o fecho convexo de uma nuvem de dez pontos pelos algoritmos de Jarvis e Graham, informa a quantidade de vértices, a área, o perímetro e o diâmetro do fecho.
- **demo_spatial:** encontra o par de pontos mais próximos da mesma nuvem usada em “demo_convex_hull” e analisa a interseção entre dois triângulos, informando a quantidade de pontos de interseção e de regiões de interseção.

Ao fim da execução de “main.c”, as memórias usadas são liberadas a partir das funções “free_polygon” e “free”.

Segue abaixo o resultado gráfico da execução:



5 Makefile

O arquivo Makefile do projeto, localizado na base do repositório, é responsável por compilar a biblioteca (estática e dinâmica, mais detalhe na seção 6), a aplicação cliente e os testes, além de executar e instalar o projeto. Dividiremos o Makefile em algumas partes específicas.

5.1 Variáveis de Configuração

5.1.1 CC e CFLAGS: compilador “gcc” com “-std=c11 -Wall -Wextra -pedantic -O2 -Iinclude” configura a compilação do projeto, ativando os avisos que permitem a detecção de erro e a otimização nível 2.

5.1.2 DEPFLAGS: “-MMD -MP” permite a geração de arquivo de dependência “.d”, fundamental para compilação incremental.

5.1.3 LDLIBS: “-lm” liga a biblioteca matemática.

5.1.4 PREFIX: indica o diretório de instalação, derivando as linhas seguintes LIBDIR e INCLUDEDIR.

5.1.5 diretórios: são os diretórios que armazenam cada tipo de arquivo necessário durante a compilação do projeto, como “src/”, “include/”, “app/”, “tests/”, “build/”, “lib/” e “bin/”.

5.2 Regras

Para evitar erros durante o processo de compilação, algumas regras foram estabelecidas.

5.2.1 .PHONY: evita que os comandos sejam interpretados como arquivos. Os comandos adicionados nessa regra são:

- all;
- run;
- run-static;
- run-shared;
- test;
- install;
- uninstall;
- clean;
- help.

Cada comando será tratado separadamente na seção 5.3.

5.2.2 .SECONDARY: evita a deleção dos objetos de teste, por serem intermediários.

5.3 Alvo

Os alvos de Makefile são os nomes dos comandos que o Make deve realizar. Para o seguinte trabalho, foram estabelecidos os seguintes alvos.

5.3.1 all: alvo padrão, usado caso nenhum comando seja adicionado na execução de Make.

O alvo “all” apresenta 4 pré-requisitos, que são “libgeom.a”, “libgeom.so”, “app_static” e “app_shared”. Para o alvo “all”, tem algumas regras gerais e para pré-requisitos específicos.

5.3.1.1 build/lib/%.o: regra para compilar os objetos da biblioteca. Ela compila os objetos da biblioteca, gerando arquivos “.o” e arquivos “.d”, além de um

código necessário para biblioteca dinâmica, por meio do opção “-fPIC”. Essa regra apresenta como pré-requisito arquivos do tipo “.c” no diretório “src/”.

5.3.1.2 lib/libgeom.a: regra para biblioteca estática. Ela empacota todos os objetos gerados para formar um arquivo “.a”. Essa regra apresenta como pré-requisito todos os arquivos objeto criados.

5.3.1.3 lib/libgeom.so: regra para biblioteca dinâmica. Ela gera um arquivo “.so”, que representa a biblioteca dinâmica. Essa regra apresenta como pré-requisito os arquivos objeto criados.

5.3.1.4 build/app/%.o: geração de arquivo objeto da aplicação cliente, isto é, os arquivos “main.c” e “svg.c”, cada um gerando o seu objeto. Cada objeto tem como pré-requisito o arquivo “.c” correspondente em “app/”.

5.3.1.5 bin/app_static: link estática do programa do cliente. Ela faz o processo de ligação estática entre o programa do cliente e a biblioteca estática gerada. Ela apresenta como pré-requisito o arquivo “.a” anteriormente criado e os objetos da aplicação (“main.o” e “svg.o”).

5.3.1.6 bin/app_shared: link dinâmica do programa do cliente. Ela faz o processo de ligação dinâmica entre o programa do cliente e a biblioteca dinâmica gerada. Ela apresenta como pré-requisito o arquivo “.so” anteriormente criado e os objetos da aplicação (“main.o” e “svg.o”).

5.3.2 run: alvo de execução do programa, que deriva os dois alvos, dependendo do tipo de ligação desejada pelo cliente. Apresenta como pré-requisito os alvos abaixo.

5.3.2.1 run-static: execução do programa cliente a partir de ligação estática. Esse alvo apresenta como pré-requisito o arquivo executável “app_static”, onde apresenta o resultado da ligação estática, visto na seção 5.3.1.5.

5.3.2.2 run-shared: execução do programa cliente a partir de ligação dinâmica. Esse alvo apresenta como pré-requisito o arquivo executável “app_shared”, onde apresenta o resultado da ligação dinâmica, visto na seção 5.3.1.6.

5.3.3 test: alvo que realiza os programas de teste. Ao fim da execução, o alvo retorna se todos os testes foram realizados com sucesso ou a quantidade de arquivos de teste que falharam, terminando com código de erro. O pré-requisito deste alvo são os executáveis “bin/tests/test_*”.

5.3.4 install: alvo que realiza a instalação do projeto, copiando as bibliotecas para PREFIX/lib e os arquivos de cabeçalho para PREFIX/include/geom, com as permissões corretas. O PREFIX padrão é /usr/local e pode ser alterado para instalar sem privilégios de administrador, por exemplo com make install

PREFIX=”$HOME/.local”. O alvo apresenta como pré-requisitos as bibliotecas compiladas.

5.3.5 uninstall: alvo que remove as bibliotecas e os arquivos de cabeçalho instalados, usando o mesmo PREFIX da instalação. O alvo não apresenta pré-requisito, pois apenas remove as bibliotecas em PREFIX.

5.3.6 clean: alvo que realiza a limpeza dos artefatos criados, apagando os diretórios “build/”, “lib/” e “bin/” e seus conteúdos. O alvo não exige pré-requisito.

5.3.7 help: alvo com as informações dos alvos, além do exemplo de instalação sem privilégio. O alvo não exige pré-requisito.

5.4 Compilação Incremental

A compilação incremental é realizada a partir de dois mecanismos. Primeiro, cada arquivo do tipo “.c” presente no diretório “src/” é compilado em seu próprio objeto, por meio das regras de padrão. Dessa forma, o Make apenas compara a data de modificação de “.c” e de “.o” correspondente e identifica a mudança para realizar recompilação direcionada.

Segundo, as opções de compilação “-MMD -MP” permite a geração de arquivo de dependência do tipo “.d”, permitindo a recompilação apenas dos módulos que o utilizam. Assim, os módulos não influenciados pela mudança são reaproveitados.

6 Geração e Utilização de Bibliotecas

A geração das bibliotecas segue as regras definidas pelo Makefile, descrita na seção 5.3. Nesta seção, será mostrada a diferença entre as duas bibliotecas.

6.1 Biblioteca Estática

A geração da biblioteca estática segue a regra “ar rcs $@ $^”, que empacota todos os arquivos objeto do pré-requisito em uma única biblioteca com nome do alvo da regra, ou seja, “libgeom.a”.

Após a geração, a ligação entre a biblioteca estática “.a” e arquivo objeto cliente ocorre a partir da regra “$(CC) $(APP_OBJS) $(STATIC_LIB) $(LDLIBS) -o $@”, em que o linker copia para o executável os módulos (arquivos objetos) da biblioteca que contêm as funções necessárias.

O executável gerado a partir de biblioteca estática não depende mais da biblioteca, uma vez que toda a função necessária já está contida no executável, o que torna o executável mais pesado, porém com menor risco de falha com futuras mudanças.

No entanto, mudanças nos módulos exigem a geração de um novo executável com a nova biblioteca.

6.2 Biblioteca Dinâmica

A geração da biblioteca dinâmica segue a regra “$(CC) -shared $^ $(LDLIBS) -o $@”, com o comando “-shared” que instrui o compilador a gerar a biblioteca dinâmica “.so”.

Após a geração, a ligação entre a biblioteca dinâmica “.so” e arquivo objeto cliente ocorre a partir da regra “$(CC) $(APP_OBJS) -L$(LIB_DIR) -l$(LIB_NAME) -Wl,-rpath,‘$$ORIGIN/../$(LIB_DIR)’ $(LDLIBS) -o $@”, em que o linker faz a busca pelo arquivo “.so” dentro do diretório das bibliotecas. Diferentemente da ligação estática, o executável recebe apenas o nome da biblioteca e as referências às funções que utiliza. A opção “-fPIC”, usada na compilação dos objetos, gera código independente de posição, o que permite que a biblioteca possa ser carregada em qualquer endereço de memória e compartilhada entre processos.

O executável gerado a partir de biblioteca dinâmica depende da biblioteca, uma vez que apresenta apenas o caminho de busca. Logo, alteração de endereço da biblioteca pode resultar em falha do executável, mas permite que mudanças nos módulos não exijam a geração de um novo executável, apenas alteração na biblioteca. No entanto, essa forma de ligação permite geração de executável mais leve.

7 Testes

Os testes realizados para o projeto se localizam no diretório “tests/”, dentro do repositório. Para cada módulo criado, geramos um arquivo “.c” independente de teste para cada um deles, que será analisado nesta seção.

Todos os testes compartilham o cabeçalho de apoio “test_utils.h”, que fornece algumas utilidades essenciais:

- **checkTrue:** função que analisa se a expressão é verdadeira, em casos falsos, imprime a expressão que falhou, o arquivo e a linha onde ocorreu a falha.
- **checkNear:** função que verifica os valores em double, verificando se o resultado está dentro do limite de tolerância. Em caso de erro, imprime as expressões obtidas e esperadas, além de seus valores, e o arquivo e a linha onde o erro ocorreu.
- **ASSERT_TRUE(COND):** macro que registra falha se a condição passada não for verdadeira, a partir da função “checkTrue”.;

- **ASSERT_NEAR(obtido, esperado, eps):** macro que compara os valores em ponto flutuante (double) com a tolerância (GEOM_EPS, definido em “general.h”), a partir da função “checkNear”.
- **testReport(modulo):** função que imprime o resumo dos testes de cada módulo, retornando 0 caso todos os testes passaram ou 1 caso houve falhas.

As funções de verificação realizam contagem de testes realizados e testes falhos, permitindo a visualização dos erros.

7.1 Testes dos Módulos

7.1.1 test_general.c: os testes para o módulo “general” ocorre apenas com a verificação da precisão das coordenadas dos pontos gerados pela função “create_point” e dos pontos dos segmentos de reta gerados pela função “create_segment”.

7.1.2 test_geometry2d.c: os testes para o módulo “geometry2d” ocorre de forma direcionada para cada função.

7.1.2.1 test_point_on_segment: realizamos testes para analisar os casos de ponto médio, ponto extremos e ponto fora de segmento.

7.1.2.2 test_point_distance: realizamos testes para pontos com coordenadas positivas e negativas e pontos congruentes.

7.1.2.3 test_point_line_distance: realizamos testes para pontos acima do segmento, abaixo do segmento, sobre o segmento e para além do comprimento do segmento, para verificar se a distância é realmente sobre a reta, e não segmento de reta.

7.1.2.4 test_line_intersection: realizamos testes para retas perpendiculares e para retas em que os seus segmentos não se cruzam, para verificar se a intersecção é sobre as retas e não sobre os segmentos.

7.1.2.5 test_lines_position: realizamos testes para retas paralelas, coincidentes (caso de retas paralelas), perpendiculares e concorrentes não perpendiculares.

Além desses casos, cada função é testada com entradas inválidas: ponteiros nulos, segmentos com os dois pontos iguais e, em "line_intersection", retas paralelas. Os testes verificam os retornos de erro documentados (0, -1, INVALIDO ou NAN).

7.1.3 test_polygon: os testes para o módulo “polygon” ocorre de forma direcionada para cada função.

7.1.3.1 test_create_polygon: realizamos testes para verificar a criação de polígonos válidos e inválidos (quantidade de vértices inferior a 3 e array de pontos nulo).

7.1.3.2 test_free_polygon: realizamos testes para verificar se o polígono fica vazio após a execução da função, além dos casos degenerados (liberar duas vezes ou liberar NULL).

7.1.3.3 test_signed_area: realizamos testes para polígonos ordenados em sentido anti-horário (valor positivo) e em sentido horário (valor negativo), além de aplicar para entrada inválida.

7.1.3.4 test_polygon_area: realizamos testes para polígonos de ambos os sentidos, para verificar se o valor é sempre positivo, além de aplicar para entradas inválidas.

7.1.3.5 test_polygon_perimeter: realizamos testes para quadrado e triângulo, além de aplicar em entradas inválidas.

7.1.3.6 test_polygon_centroid: realizamos testes para triângulos, além de verificar que a obtenção de centroide não depende do sentido do polígono e aplicar em entradas inválidas.

7.1.3.7 test_is_convex: realizamos testes para polígonos convexos, em ambos os sentidos e côncavos e em polígonos com um vértice sobre uma aresta, além de aplicar em entradas inválidas.

7.1.3.8 test_polygon_contains_point: realizamos testes para pontos estritamente dentro do polígono, sobre aresta e vértice, fora dele e na reentrância de um polígono côncavo (fora do polígono), além de aplicar em entradas inválidas.

7.1.4 test_convex_hull: os testes para o módulo “convex_hull” ocorre de forma direcionada para cada função.

7.1.4.1 test_fecho: realizamos testes para verificar se o fecho criado corresponde à resposta, tanto para algoritmo Jarvis como para Graham, aplicando para triângulo, quadrado, pontos colineares (não há fecho) e entradas inválidas.

7.1.4.2 test_graham_igual_jarvis: realizamos testes para verificar se o fecho gerado pelo algoritmo de Jarvis coincide com o fecho gerado pelo algoritmo de Graham.

7.1.4.3 test_area: realizamos testes para verificar a área dos fechos gerados, além de aplicar para entradas inválidas.

7.1.4.4 test_perimetro: realizamos testes para verificar o perímetro dos fechos gerados, além de aplicar para entradas inválidas.

7.1.4.5 test_diametro: realizamos testes para verificar o diâmetro dos fechos gerados, além de aplicar para entradas inválidas.

7.1.5 test_spatial: os testes para o módulo “spatial” ocorre de forma direcionada para cada função.

7.1.5.1 test_closest_pair: realizamos testes para conjuntos com um par mais próximo, com apenas dois pontos e com pontos repetidos, além de aplicar para entradas inválidas.

7.1.5.2 test_intersection_polygon: realizamos testes para quadrados sobrepostos, com dois pontos de interseção, polígonos sem interseção e polígono contido, além de entradas inválidas (polígono vazio e ponteiro length nulo).

7.1.5.3 test_is_polygon_inside: realizamos testes para polígonos A e B, tais que A está contido em B, A não está contido em B, A é igual a B e A não totalmente contido em B, além de polígono vazio.

7.1.5.4 test_intersection_region: realizamos testes para quadrados sobrepostos (região de interseção é um quadrado menor), polígono contido no outro, polígonos sem interseção, dois triângulos cruzados em forma de estrela, cuja interseção é um hexágono de área 12, e entradas inválidas

7.2 Estratégia de Teste

Os testes foram pensados de forma a englobar as situações mais comuns de aplicação, além das situações mais extremas, mas possíveis de ocorrer. Como os testes foram separados em módulos, torna-se mais fácil localizar o erro.

Os casos de teste são executados a partir do comando “make test”, em que todos os testes são executados após a compilação e a ligação dos programas. Além disso, o resultado de cada teste é impresso para o usuário, como definido em Makefile (seção 5).

7.3 Resultado

A execução de make test compila e executa os cinco arquivos de teste, com 167 verificações no total (general 10, geometry2d 39, polygon 43, convex_hull 27 e spatial 48), e todas passam na versão final. Durante o desenvolvimento, porém, alguns testes falharam e levaram a correções no código. Um desses casos é descrito na seção 7.4.

7.4 Teste que falhou inicialmente

Ao escrever os testes do módulo "polygon" (commit 66f54e4, branch "Testes"), incluímos o quadrado de vértices (0, 0), (2, 0), (4, 0), (4, 4) e (0, 4). Ele é convexo,

mas tem um vértice no meio de uma aresta. O teste ASSERT_TRUE(is_convex(&p) == 1) falhou, e make test indicou [polygon] 41/42 testes passaram -> FALHA.

O motivo estava no primeiro trio de vértices, (0, 0), (2, 0) e (4, 0). Por ser colinear, o produto vetorial é zero, e a função "is_convex" tratava esse zero como uma curva no sentido horário. Quando encontrava as curvas seguintes, todas no sentido anti-horário, concluía que o polígono não era convexo.

A correção (commit b578043) faz a função ignorar trios colineares, sem alterar a orientação: if (fabs(cross) < GEOM_EPS) continue;. Depois dela, todos os testes passaram ([polygon] 42/42), e a branch "Testes" foi integrada ao "main" pelo merge 81479da. O caso pode ser reproduzido com git switch --detach 66f54e4 && make test.

8 Versionamento

O controle de versão do projeto foi realizado por meio da ferramenta Git, na plataforma GitHub. O repositório criado pode ser acessado por todos os integrantes do grupo para realizar as modificações relativas a seu trabalho.

Para cada módulo principal (4) citado anteriormente, foi criado um branch separado, administrado por cada integrante responsável, para o desenvolvimento das funções específicas. Após a conclusão dos módulos, o merge é realizado com o consentimento do grupo.

Com isso, é possível o controle do desenvolvimento do trabalho, além de compartilhamento das situações de desenvolvimento.

8.1 Branch

Ao longo do desenvolvimento do projeto, foi criado um branch para cada módulo implementado, além de branch extras para estruturar o projeto, sendo eles:

8.1.1 GeometriaPlana: branch criado para implementação do módulo “geometry2d”.

8.1.2 Poligono: branch criado para implementação do módulo “polygon”.

8.1.3 EnvoltoriaConvexa: branch criado para implementação do módulo “convex_hull”

8.1.4 IntersecaoProximidade: branch criado para implementação do módulo “spatial”.

8.1.5 feature/sistema-de-build: branch criado para padronizar a estrutura do projeto, com a criação dos diretórios e arquivos necessários.

8.1.6 Padronizacao: branch criado para padronizar os nomes das funções.

8.1.7 Testes: branch criado para implementação dos casos de teste.

8.1.8 Correcoes: branch criado para corrigir erros e vazamentos de memória em alguns módulos.

8.1.9 feature/aplicacao-cliente: branch criado para implementar a aplicação cliente com geração de figuras SVG.

8.2 Merge

O merge de um branch é realizado quando a implementação do módulo, ou a estruturação do projeto, está concluída. As branches foram integradas ao "main" por commits de merge, em geral com git merge --no-ff, que cria o commit de merge mesmo quando seria possível apenas avançar o "main". Assim, o histórico (git log --graph) mostra onde cada funcionalidade começou e terminou. Alguns exemplos são e8d8fad (feature/sistema-de-build), 0dd7f17 (EnvoltoriaConvexa), 5ddcfad (IntersecaoProximidade), 81479da (Testes), 44ff51a (Correcoes) e 30cee48 (feature/aplicacao-cliente).

Como o módulo "spatial" depende de "geometry2d" e "polygon", a branch IntersecaoProximidade incorporou as branches desses módulos antes de ser concluída (merges 6c48261 e ff888df).

8.3 Tag v1.0

A tag anotada v1.0 foi criada com git tag -a v1.0 sobre o commit final do projeto, em que make, make test e make run funcionam, e enviada ao GitHub com git push origin v1.0. Essa versão pode ser obtida com git checkout v1.0.

9 Decisões

Ao longo do desenvolvimento do projeto, algumas decisões foram tomadas para garantir o funcionamento dos programas.

9.1 GEOM_EPS

Inicialmente, a constante se chamava EPS e era declarada como float (1e-6f). O nome genérico entrava em conflito com outros códigos que também definem uma constante EPS, por isso recebeu o prefixo da biblioteca, e o valor passou a ser double.

O módulo "convex_hull" é uma exceção: suas comparações são exatas. O algoritmo só precisa reconhecer pontos idênticos, e uma tolerância aplicada a apenas parte das comparações fez as funções de Graham e de Jarvis discordarem em conjuntos

com pontos muito próximos. Por isso, mantivemos a comparação exata nesse módulo.

9.2 general

Há algumas estruturas e funções que são usadas em quase todos os módulos implementados, como “Ponto”, “Segmento”, “create_point” e “create_segment”. Por esse motivo, para evitar a criação de estruturas e funções repetidas em todos os módulos, optamos por criar um módulo base, “general”, para englobar tais utilidades essenciais.

9.3 Testes

A decisão de criar arquivos separados de teste para cada módulo é para evitar a confusão durante a busca pelos erros, permitindo a localização direto do módulo a ser corrigido. Além disso, a criação de um arquivo de cabeçalho “test_utils.h” tem como finalidade a padronização dos testes.

9.4 Aplicação Cliente

O desenvolvimento da aplicação cliente com geração de imagem tem como objetivo facilitar a visualização dos programas, uma vez que os problemas envolvem geometria.

9.5 Geração de Fecho Convexo

A escolha por implementar duas funções para criação de fecho convexo no módulo “convex_hull” é devido à diferença de complexidade de cada algoritmo, o que permite ao cliente decidir a função que será utilizada, a partir de suas entradas.

10 Dificuldades

Ao longo do desenvolvimento do projeto, algumas dificuldades relevantes foram encontrados, como:

- Ligação entre funções de diferentes módulos, uma vez que os módulos foram desenvolvidos por cada integrante de forma isolada.
- Escolha de testes que engloba todos os casos possíveis, garantindo a funcionalidade adequada das funções.
- Criação do arquivo Makefile que cumpra todos os requisitos apresentados.

11 Resultados e Possíveis Melhorias

O trabalho conseguiu alcançar o seu objetivo de resolver os problemas envolvendo geometria a partir dos módulos “geometry2d”, “polygon”, “convex_hull” e “spatial” e suas funções. No entanto, o projeto ainda apresenta alguns pontos de melhoria, listados abaixo.

- Reduzir a complexidade de tempo de certas funções, para garantir a maior eficiência na execução do programa.
- Adicionar algoritmos de verificação de pré-requisitos, evitando depender das entradas do cliente.
- Implementação de mais funções, a fim de englobar mais temas de geometria.

Esperamos, que nas versões futuras do projeto, possamos aplicar todas as melhorias levantadas acima para tornar o projeto mais completo.